WEEK 2 · LESSON 10 OF 24 · HANDOUT

Support Vector Machines: The Widest Street

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=efR1C6CvhmE — open the lesson online to watch inside the course.

🎓 **Go deeper:** Harvard CS50 AI, Lecture 4: Learning (supervised, reinforcement, unsupervised) — https://www.youtube.com/watch?v=E4M_IQG0d9g

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 1 (scikit-learn core developers) — https://www.youtube.com/watch?v=4PXAztQtoTg

Your goal in this lesson: Explain margins, support vectors and kernels, train an SVM in scikit-learn, and tune its C setting.

The widest possible street

Imagine borrowers drawn as dots on a map, repaid in blue and defaulted in red. Many lines could separate the two colours. A **support vector machine (SVM)** chooses the line that leaves the **widest street** (the **margin**) between the classes. A wide margin means new points near the boundary are less likely to be misclassified.

The few points that touch the edges of the street are the **support vectors** — they alone decide where the line goes. Points far from the boundary do not matter at all.

Soft margins and C

Real data overlaps, so a perfect street is impossible. The setting **C** decides how strict the SVM is:

- **Small C** — a wide street that allows some points inside it or on the wrong side. Simpler, can underfit.
- **Large C** — a narrow street that tries hard to classify every training point. Can overfit.

Curved boundaries: the kernel trick

Some patterns cannot be split by a straight line. A **kernel** lets the SVM work as if the data were moved into more dimensions, where a flat boundary does exist — without actually computing those dimensions. The default **RBF kernel** gives smooth curved boundaries.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

Xm, ym = make_moons(n_samples=400, noise=0.25, random_state=0)
Xm_tr, Xm_te, ym_tr, ym_te = train_test_split(Xm, ym, random_state=0)

print("Logistic regression:", LogisticRegression().fit(Xm_tr, ym_tr).score(Xm_te, ym_te))
print("Linear SVM:         ", SVC(kernel="linear").fit(Xm_tr, ym_tr).score(Xm_te, ym_te))
print("RBF SVM:            ", SVC(kernel="rbf").fit(Xm_tr, ym_tr).score(Xm_te, ym_te))
```

The "moons" are two curved shapes. Straight-line models score about **0.89–0.90**; the RBF kernel bends around the moons and reaches about **0.93**.

Draw the boundary

```
import numpy as np
import matplotlib.pyplot as plt

svm = SVC(kernel="rbf", C=1).fit(Xm_tr, ym_tr)
xx, yy = np.meshgrid(np.linspace(-2, 3, 300), np.linspace(-1.5, 2, 300))
zz = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.contourf(xx, yy, zz, alpha=0.3)
plt.scatter(Xm[:, 0], Xm[:, 1], c=ym, s=12)
plt.title("RBF SVM decision boundary")
plt.show()
```

SVM on the loans

SVMs measure distances, so **scaling is required**.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

for C in [0.1, 1, 10, 100]:
    s = SVC(C=C).fit(X_train_s, y_train)
    print("C =", C, "train", round(s.score(X_train_s, y_train), 3),
          "test", round(s.score(X_test_s, y_test), 3))
```

As C grows, training accuracy rises steadily (from about 0.84 to 0.89) but the test score peaks around C = 10 (about **0.85**) and then falls back — the overfitting pattern again.

Probabilities and speed

- `SVC` gives labels by default. Use `SVC(probability=True)` if you need probabilities (slower).
- Training time grows quickly with the number of rows. For more than about 50,000 rows, prefer `LinearSVC` or a different model.

When to use an SVM

Good for medium-sized data with many features (for example text or gene data), and when the boundary is curved but smooth. Less good for very large data, or when you must explain each decision.

Key terms

- **Margin** — the gap between the boundary and the closest points of each class.
- **Support vectors** — the points on the edge of the margin that define the boundary.
- **C** — how strongly mistakes on training data are punished.
- **Kernel** — a function that lets SVMs draw curved boundaries (linear, RBF, polynomial).
- **Kernel trick** — working in a higher-dimensional space without computing it.

Remember this: An SVM finds the widest street between classes; kernels let that street curve, C sets how strict it is, and features must be scaled.

Name: _____ Date: _____

Activity

Task 1

Draw the moons boundary for C = 0.1 and C = 100. Describe the difference.

Task 2

Try `SVC(kernel="poly", degree=3)` on the moons. Better or worse than RBF?

Task 3

Train an RBF SVM on the **unscaled** loans data. What happens to the test accuracy?

Task 4

Print `svm.n_support_` for the moons model. How many support vectors does each class have?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does an SVM try to maximise?

   A. The training time
   B. The number of features
   C. The number of trees
   D. The margin between the classes

2. What are support vectors?

   A. The test points
   B. The features
   C. All the training points
   D. The points on the edge of the margin that define the boundary

3. What does a very large C do?

   A. Tries hard to classify every training point, risking overfitting
   B. Allows many mistakes with a wide street
   C. Scales the data
   D. Removes the kernel

4. Why use the RBF kernel on the moons data?

   A. The classes are separated by a curved boundary
   B. It needs no scaling
   C. The data has no labels
   D. It is faster

5. Must features be scaled for an SVM?

   A. Only for regression
   B. Yes, because SVMs use distances
   C. No
   D. Only for linear kernels

6. When is an SVM a poor choice?

   A. Hundreds of thousands of rows
   B. Text data
   C. Medium data with many features
   D. Curved boundaries

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.